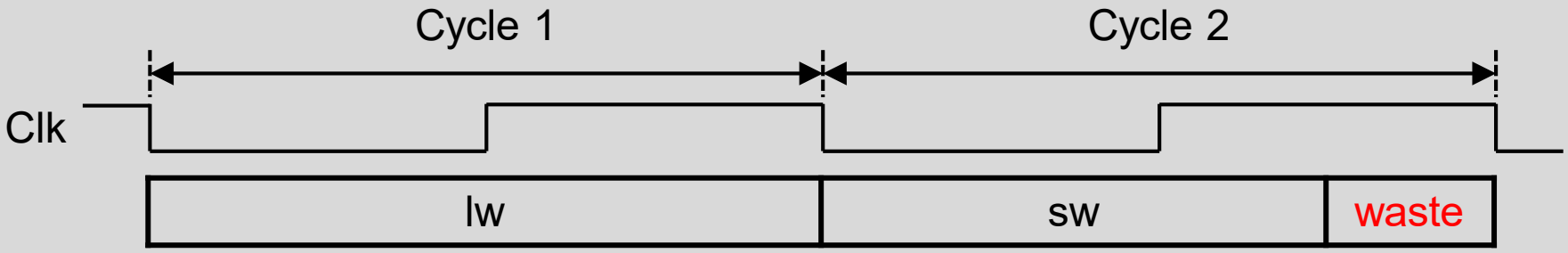
Computer System Architecture

Lecture 6 - Pipelining

November 2026

Review: Single-Cycle Disadvantages & Advantages

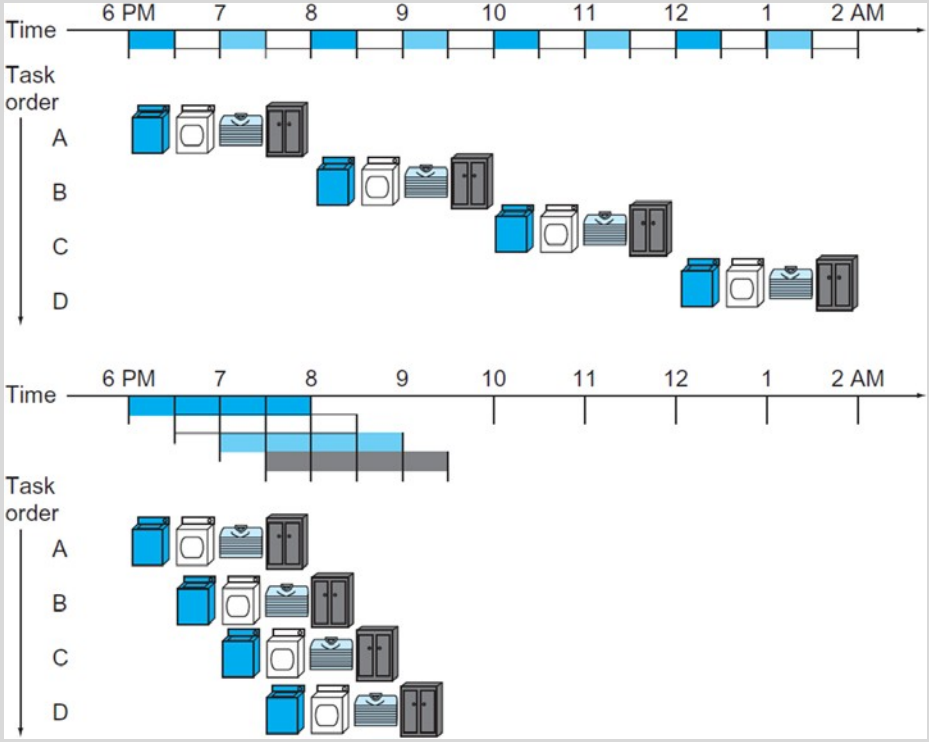
- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the **slowest** instruction
  - especially problematic for more complex instructions, e.g., floating-point multiplication



- May be wasteful of area since some functional units (e.g., adders) must be duplicated since they can not be shared during a clock cycle
- But it is simple and easy to understand

Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance
  - 4-load speedup:
    = 8/3.5
    = 2.3
  - 20-load speedup:
    = 40/11.5
    = 3.478
- Under ideal conditions
  - The speedup from pipelining ≈
    The number of pipe stages



Five Stages of Load Instruction

- Five stages, one step per stage

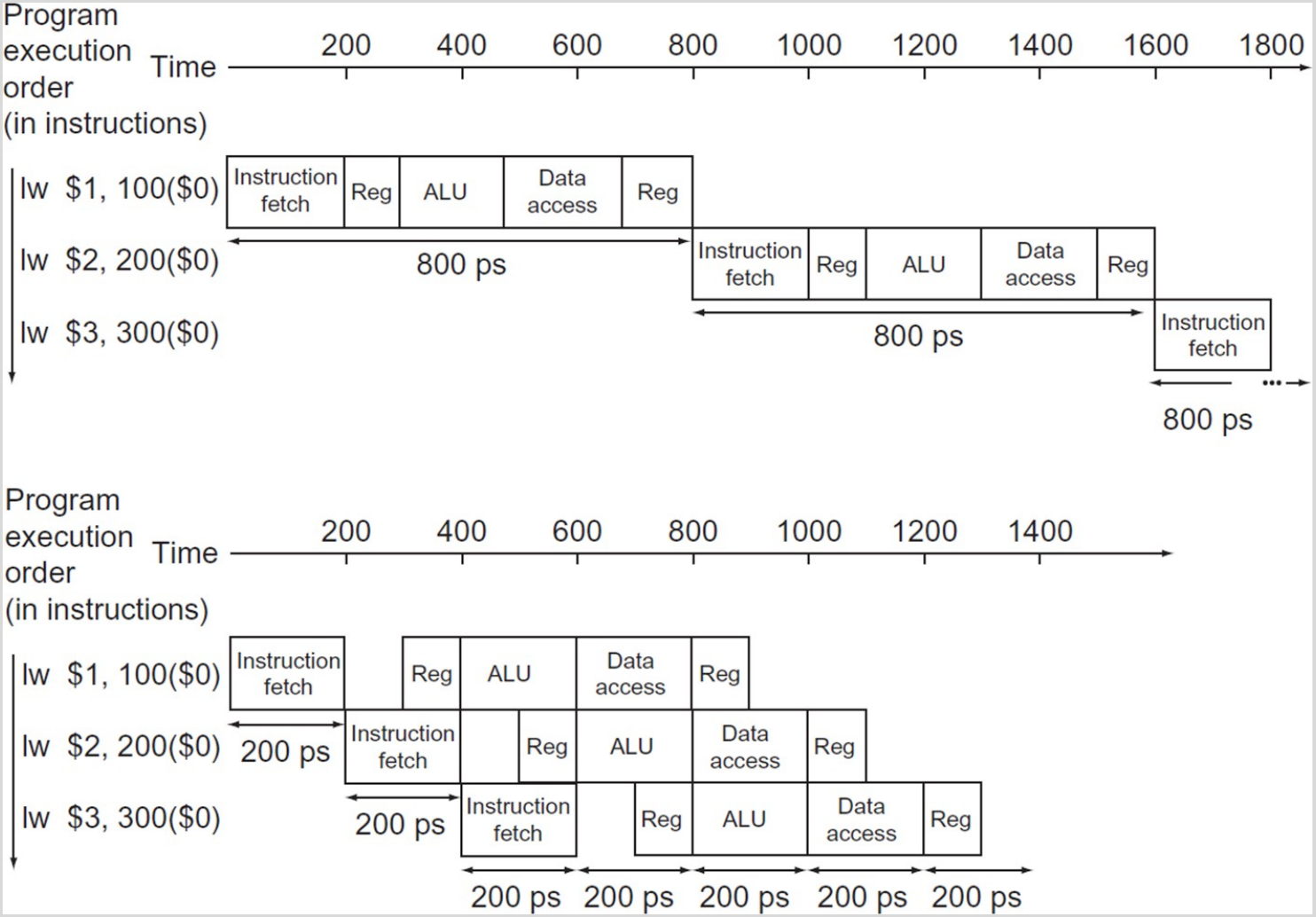
1. **IF**: Instruction fetch from (instruction) memory
2. **ID**: Instruction decode & register read
3. **EX**: Execute operation or calculate address
4. **MEM**: Access (data) memory operand
5. **WB**: Write result back to register

Pipeline Performance

- Assume time for stages is
  - 100 ps for register read or write
  - 200 ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instruction	IF	Reg Read	ALU Op	Mem Access	Reg Write	Total
lw	200	100	200	200	100	**800**
sw	200	100	200	200		700
R-format	200	100	200		100	600
beq	200	100	200			500

Pipeline Performance



Pipeline Speedup

- If all stages are balanced
- i.e., all take the same time

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

- If not balanced, speedup is less
- Speedup due to increased **throughput**
  - **Latency** does not decrease

Pipelining and ISA Design

- MIPS ISA designed for pipelining
  - All **instructions** are **32-bits**
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and **regular** instruction **formats**
    - Can decode and read registers in one step
  - Load/store addressing
    - Can **calculate address** in 3rd stage, **access memory** in 4th stage
  - Alignment of **memory operands**
    - Memory access takes only one cycle

Hazards

- Situations that prevent starting the next instruction in the next cycle
- **Structure hazards**
  - A required resource is busy
- **Data hazard**
  - Need to wait for previous instruction to complete its data read/write
- **Control hazard**
  - Deciding on control action depends on previous instruction

Structure Hazards

- Conflict for use of a resource
- In MIPS pipeline with a **single memory**
  - **Load/store** requires data access
  - **Instruction fetch** would have to ***stall*** for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require **separate** instruction/data memories
  - Or separate instruction/data caches

Data Hazards

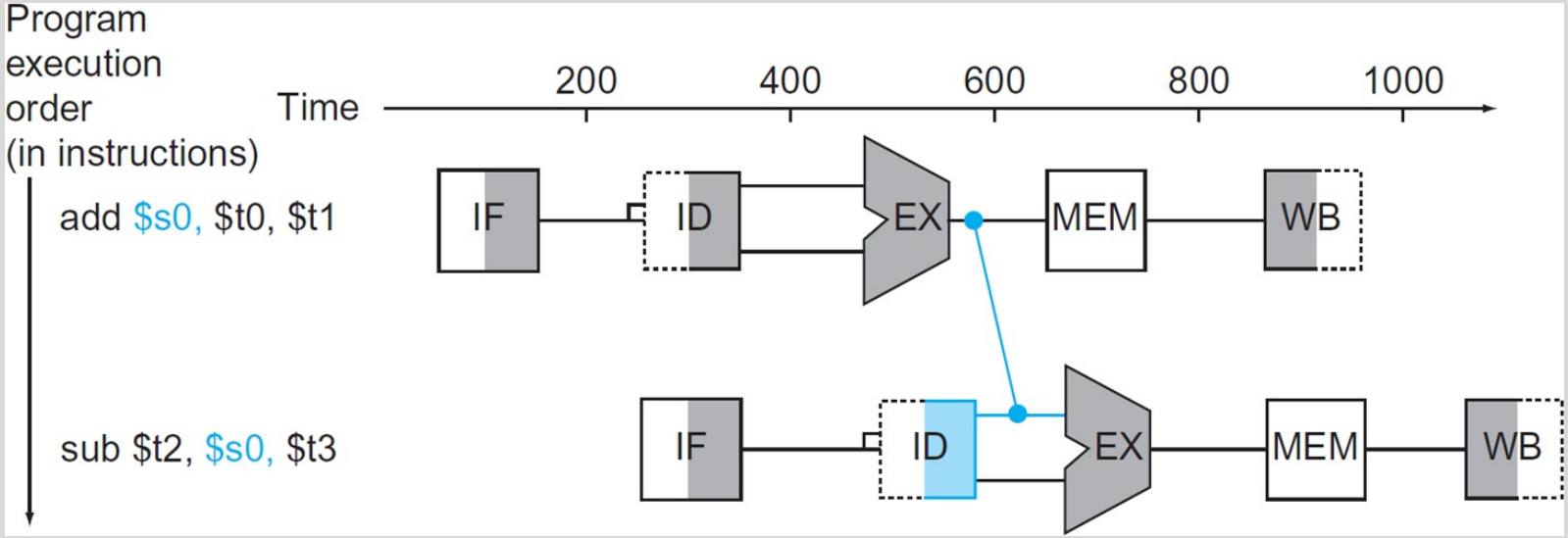
- An instruction depends on completion of data access by a previous instruction

```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

- We may need to insert a **pipeline stall** (or **bubble**).

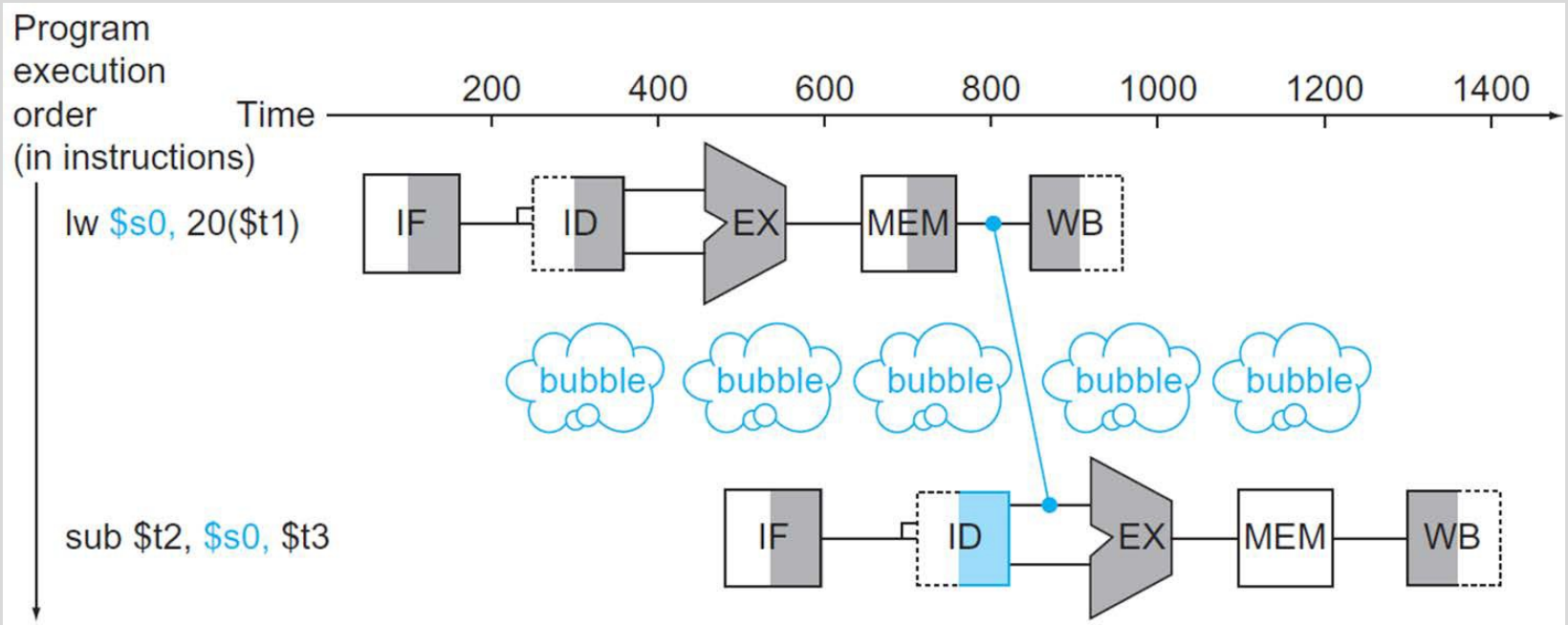
Forwarding (or Bypassing)

- Use result when it is computed
  - Don’t wait for it to be stored in a register
  - Requires **extra connections** in the datapath



Load-Use Data Hazard

- Can't always avoid **stalls** by forwarding
  - If value not computed when needed
  - Can't forward backward in time!



Reordering Code to Avoid Pipeline Stalls

- **Reorder** code to avoid use of load result in the next instruction
- Consider the C-code:

```
a = b + e; c = b + f;
```

```
        lw  $t1, 0($t0)
        lw  $t2, 4($t0)
stall   add $t3, $t1, $t2
        sw  $t3, 12($t0)
        lw  $t4, 8($t0)
stall   add $t5, $t1, $t4
        sw  $t5, 16($t0)
```

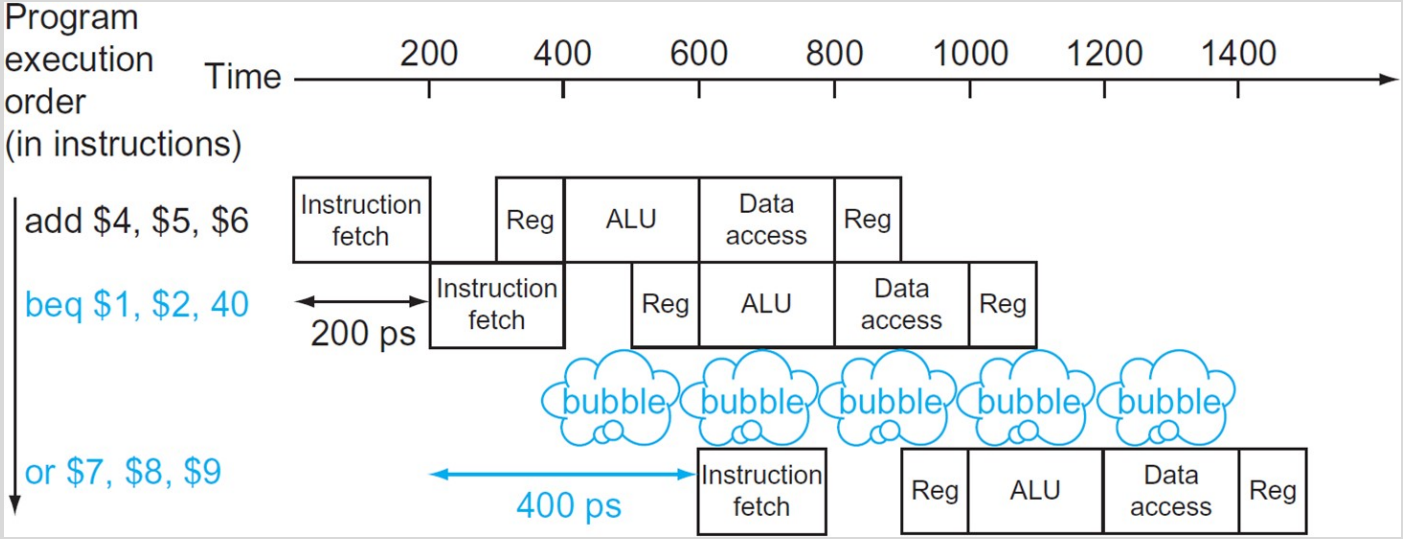
```
        lw  $t1, 0($t0)
        lw  $t2, 4($t0)
        lw  $t4, 8($t0)
        add $t3, $t1, $t2
        sw  $t3, 12($t0)
        add $t5, $t1, $t4
        sw  $t5, 16($t0)
```

Control Hazards

- **Branch** determines flow of control
  - **Fetching** next instruction depends on **branch outcome**
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Stall on Branch

- Wait until branch outcome determined before fetching next instruction
  - **Conservative approach**: Stall immediately after fetching a branch, wait until outcome of branch is known and fetch branch address.



Branch Prediction

- Longer pipelines cannot readily determine branch outcome early
  - Stall penalty becomes unacceptable
- **Predict** outcome of branch
  - Only **stall** if prediction is **wrong**
- In MIPS pipeline
  - Can predict branches **not taken**
  - Fetch instruction after branch, with no delay

Predicting Branches Are Not Taken

Prediction Correct
- **No penalty**

Prediction Incorrect
- **Stall**



More-Realistic Branch Prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict **backward** branches **taken**
    - Predict **forward** branches **not taken**
- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

Pipeline Big-Picture Summary

- Pipelining improves performance by increasing instruction **throughput**
  - Executes **multiple** instructions in **parallel**
  - Each instruction has the same **latency**
- Subject to **hazards**
  - **Structure**, **data**, **control**
- Instruction set design affects complexity of pipeline implementation